

Application Pratique : Enregistrements & Fichiers

On se propose de créer un programme **Python modulaire** qui permet :

- 1) de créer et remplir un fichier ("**d:\eleves.dat**") par des enregistrements. Un enregistrement est formé par le nom de l'élève (chaîne alphabétique pouvant contenir le caractère espace) et sa moyenne (réel de **0** à **20**)

NB. : Le nombre d'élèves est inconnu, ainsi après la saisie de chaque enregistrement, le message suivant sera affiché "**Autre saisie (O/N) ?** ", l'utilisateur doit saisir sa réponse ("**O**", "**o**", "**N**", "**n**") pour décider de continuer en répondant par ("**O**" ou "**o**") ou d'arrêter la saisie en répondant par ("**N**" ou "**n**")

- 2) d'afficher les noms des élèves comme le montre l'exemple suivant :

```
1 : Ali Mrabet : 12.33
2 : Wafa Makni : 8.25
3 : Salmen Dammak : 10.25
4 : Amina Hriz : 9.33
```

- 3) de créer un fichier texte ("**d:\admis.txt**") comportant dans sa première ligne le message : "**Les admis sont :** " puis les noms des élèves admis suivies, chacun, par sa moyenne séparés par le caractère " : " (un élève par ligne) et dans la dernière ligne du fichier le nombre d'élèves admis précédé par le message "**Nombre d'admis =** " comme le montre l'exemple ci-après :

Exemple :

⇨ Contenu du fichier "**d:\admis.txt**"

```
Les admis sont :
Ali Mrabet : 12.33
Salmen Dammak : 10.25
Nombre d'admis = 2
```

NB. : Les élèves admis sont ceux qui ont une moyenne ≥ 10

- 4) d'ajouter à la fin du fichier ("**d:\admis.txt**") le message "**Le(s) meilleur(s) élève(s) :** " suivi par le nom de l'élève (ou les noms des élèves) ayant la meilleure moyenne (un élève par ligne).

Application Pratique : Cryptage des messages

Pour sécuriser l'envoi des messages, deux chercheurs se mettent d'accord de crypter leurs correspondances en utilisant une clé de cryptage selon le principe suivant :

1. Saisir le message à crypter, sachant qu'il est composé par des lettres majuscules et pouvant contenir le caractère espace.
2. Saisir la clé de cryptage qui est une chaîne numérique ayant la même longueur que celle du message à crypter.
3. Chaque lettre du message sera cryptée de la façon suivante :
 - Remplacer chaque lettre du message d'ordre **i**, par la lettre d'ordre **j** avec $j=i+c$, sachant que **c** est le chiffre de la clé ayant le même indice que la lettre à crypter.

NB. :

- L'espace ne sera pas crypté
- Si **j** dépasse **26**, on reprend les lettres alphabétiques dès le début.

Exemple : Soit le message à crypter : "EXCELLENTE PERFORMANCE" et la clé : "1954632738401653628451"

Message à crypter	E	X	C	E	L	L	E	N	T	E	 	P	E	R	F	O	R	M	A	N	C	E
Clé de cryptage	1	9	5	4	6	3	2	7	3	8	4	0	1	6	5	3	6	2	8	4	5	1
Crypter chaque mot	F	G	H	I	R	O	G	U	W	M	 	P	F	X	K	R	X	O	I	R	H	F

En effet :

- La lettre "**E**" est d'ordre alphabétique **5**, elle sera remplacée par la lettre d'ordre alphabétique **5+1=6** c'est à dire "**F**"
- La lettre "**X**" est d'ordre alphabétique **24**, elle sera remplacée par la lettre d'ordre alphabétique **24+9=33**, **33 > 26** on reprend les lettres dès le début l'équivalent de **33 est 7** c'est à dire "**G**"
- etc...

D'où le message crypté suivant : "FGHIROGUWM PFXKR XOIRHF"

Travail demandé :

- a. En utilisant l'éditeur de texte disponible (**Bloc-notes, Notepad, Notepad++...**) saisir les lignes suivantes dans un fichier à sauvegarder sous le nom "**d:\message.txt**"

EXCELLENTE PERFORMANCE
BAC INFO
EPREUVES PRATIQUES
BON COURAGE MES ELEVES

- b. Ecrire un programme **Python** modulaire permettant de crypter les lignes du fichier "**d:\message.txt**" dans un fichier "**d:\cryptage.txt**" comme suit :

► Pour chaque ligne du fichier "**d:\message.txt**"

- saisir la clé de cryptage **chcle** en respectant la contrainte décrite précédemment.
- sauvegarder dans le fichier "**d:\cryptage.txt**" le message à crypter, la clé de cryptage et le message crypté qui lui correspond en appliquant la méthode de cryptage décrite précédemment et ceci en les séparant par le caractère "#"

Exemple :

EXCELLENTE PERFORMANCE#1954632738401653628451#FGHIROGUWM PFXKR XOIRHF

Application : Matrice et Fichiers

Soit **M** une matrice de (**n** x **m**) entiers ($2 \leq n \leq m \leq 10$). Un élément de **M** est considéré comme « **point-aigus** » s'il est à la fois un maximum sur sa ligne et un minimum sur sa colonne.

On se propose de chercher dans la matrice **M** les éléments appelés « **points-aigus** ». Pour chaque élément trouvé on donne aussi bien sa position (ses coordonnées) dans la matrice que sa valeur (voir exemples). Si aucun « **point-aigus** » n'est trouvé, on donne le message : "**aucun point-aigus trouvé**"

Exemples : Pour **n** = 3 ; **m** = 4 et les matrices **M** suivantes :

Exemple 1

	0	1	2	3
0	1	8	3	4
1	3	9	2	3
2	6	7	2	7

$M[2,1] = 7$ est un point-aigus

Exemple 2

	0	1	2	3
0	4	5	8	9
1	3	8	9	3
2	3	4	9	3

aucun point-aigus trouvé

Exemple 3

	0	1	2	3
0	4	5	6	6
1	2	2	8	9
2	3	2	9	6

$M[0,2] = 6$ est un point-aigus

$M[0,3] = 6$ est un point-aigus

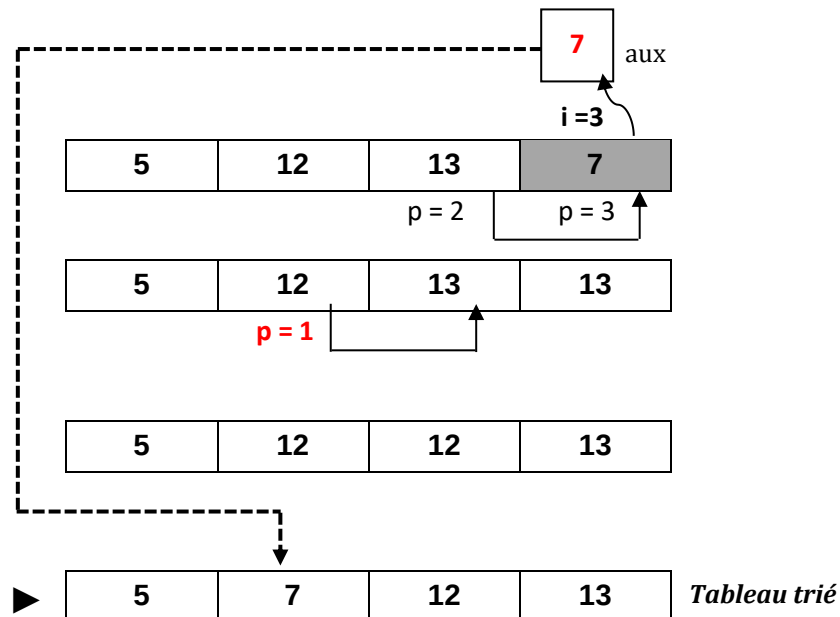
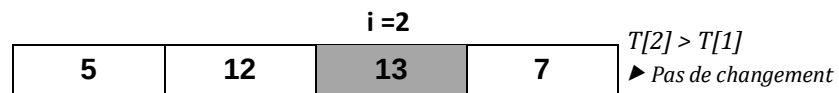
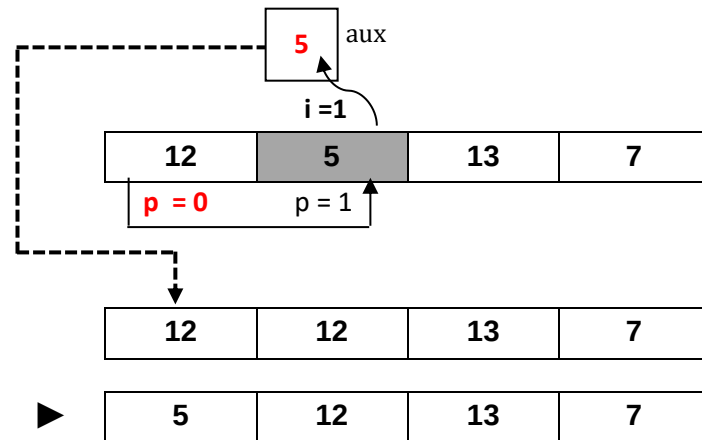
Travail demandé

- 1) Ecrire un algorithme d'une fonction **Max_Ligne(M, L, m)** qui retourne l'entier maximum dans une ligne d'indice **L** de la matrice **M**
- 2) Ecrire un algorithme d'une fonction **Min_Colonne(M, C, n)** qui retourne l'entier minimum dans une colonne d'indice **C** de la matrice **M**
- 3) Ecrire un algorithme d'une procédure **Ranger(M, n, m, F)** qui utilise les fonctions **Max_Ligne** et **Min_Colonne** pour déterminer et enregistrer dans un fichier texte **F** ("PointsAigus.txt") les « **points-aigus** » de la matrice **M** conformément à l'exemple donné ci-dessus.

3)- Tri par insertion (Ordre croissant) :

Comme son nom l'indique, cette méthode de tri consiste à :

- Insérer un élément d'indice i dans sa bonne position parmi les $i-1$ éléments précédents déjà triés de telle sorte que ces éléments restent triés.
- Répéter la même opération précédente à partir du 2^{ème} élément du tableau jusqu'au dernier (pour i de 1 à $N-1$)



Procédure Tri_insertion (.....)

Début

.....

.....

.....

.....

.....

.....

.....

Fin